

Firestore Part II

Materials Needed:

- A computer with Flutter installed.
- A code editor (like VS Code or Android Studio).
- The provider package installed in your Flutter project.

Instructions:

Use the previous code [Lab Activity] before starting.
Replicate the code snippet | Update the previous code provided.
Follow and implement the enhancement instructions.

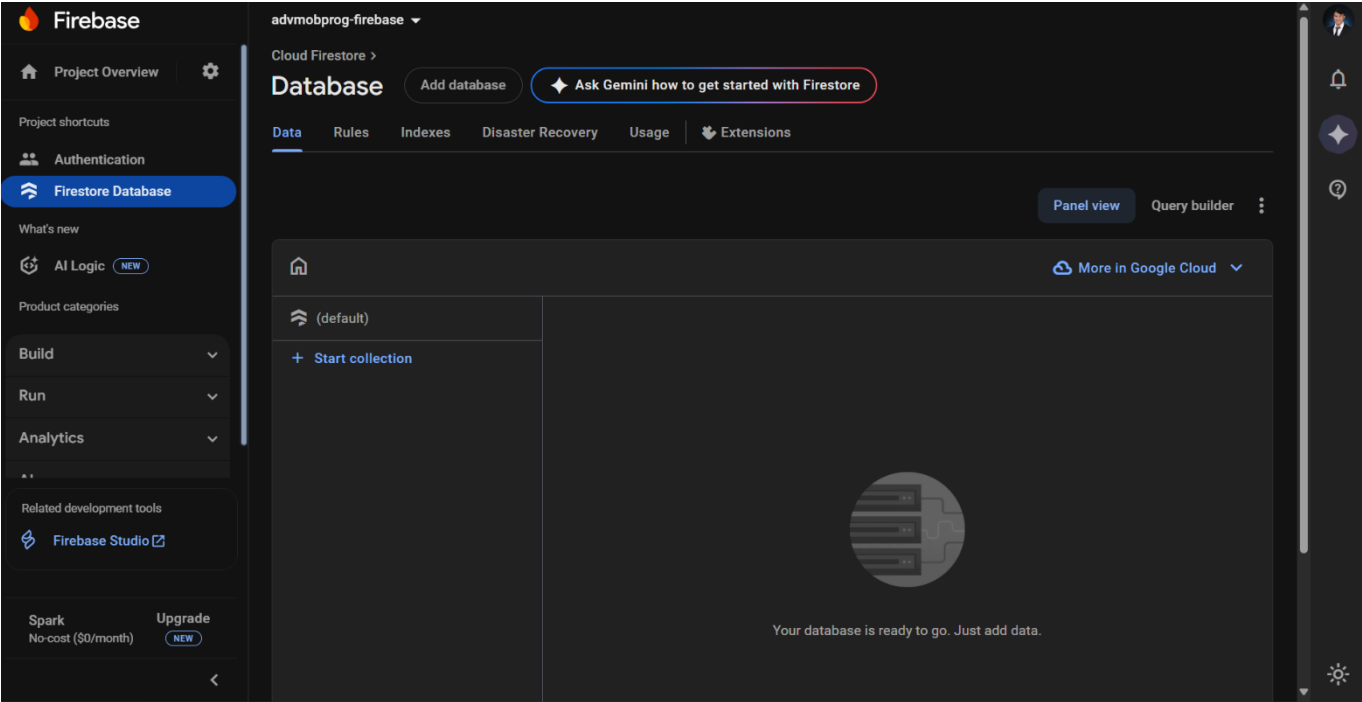
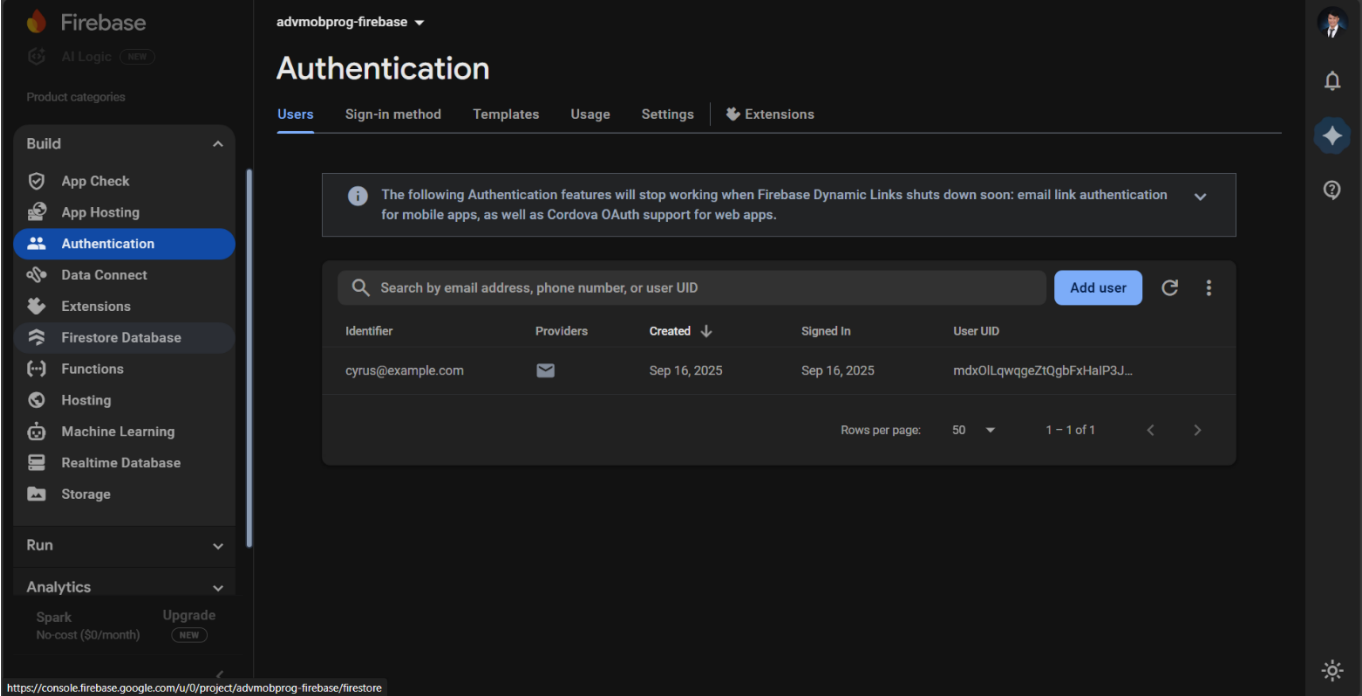
- 1) Go to <https://console.firebase.google.com/>
1. Go to build and select **Firestore Database** and select get started.

2. Select **US Region** and,

3. Select **Production Mode**,

4. Go to **Rules** tab and set the *read* and *write* to **true**

5. Go to VS Code’s terminal / flutter project and run this command: **flutter pub add cloud_firestore**





advmobprog-firebase

Cloud Firestore > Database

Project Overview

Project shortcuts

Authentication

Firestore Database

What's new

AI Logic

Product categories

Build

Run

Analytics

Related development tools

Firestore Studio

Spark

No-cost (\$0/month)

Upgrade

NEW

Guard your data with rules that define who has access to it and how it is structured

[View the docs](#)

rules_version = '2';

service cloud.firestore {

match /databases/{database}/documents {

match /{document=**} {

allow read, write: if true;

}

}

}

Rules Playground

Experiment and explore with Security Rules

Published changes can take up to a minute to propagate

Your security rules are defined as public, so anyone can steal, modify, or delete data in your database

[Learn more](#)

robles_mobile

user_service.dart U profile_screen.dart U chat_screen.dart U chat_service.dart U login_screen.dart U

lib > services > user_service.dart > UserService

```
7
8 ValueNotifier<UserService> userService = ValueNotifier(UserService());
9
10 class UserService {
11   *Map<String, dynamic> data = {};
12   final FirebaseAuth firebaseAuth = FirebaseAuth.instance;
13
14   User? get currentUser => firebaseAuth.currentUser;
15
16   Stream<User?> get authStateChanges => firebaseAuth.authStateChanges();
17
18   Future<Map<String, dynamic>> loginUser(
19     String email,
20     String password, {
21     bool useFirebase = true,
22   }) async {
23     if (useFirebase) {
24       try {
25         // Log in to Firebase
26         UserCredential userCredential = await firebaseAuth
27           .signInWithEmailAndPassword(email: email, password: password);
28
29         User? user = userCredential.user;
30         if (user != null) {
31           // Log in to HTTP backend
32           final response = await post(
33             Uri.parse('$host/api/users/login'),
34             body: {"email": email, "password": password},
35           );
36
37           if (response.statusCode == 200) {
38             data = jsonDecode(response.body);
39
40             // Save user data to SharedPreferences
```

PROBLEMS 14

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

QUERY RESULTS

POSTMAN CONSOLE

bash

ACER@LAPTOP-QVL3RPM MINGW64 ~/Desktop/cy.dev/cy.flutter/02_course_materials/robles_advmobprog/robles_mobile (main)

\$ flutter pub add cloud_firestore



Source Code

services/chat_service.dart

```
import 'package:cloud_firestore/cloud_firestore.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'package:robles_advmobprog/models/message_model.dart';

class ChatService {
  final FirebaseFirestore _firestore = FirebaseFirestore.instance;
  final FirebaseAuth _firebaseAuth = FirebaseAuth.instance;
  // get all users
  Stream<List<Map<String, dynamic>>> getUsersStream() {
    return _firestore.collection("Users").snapshots().map((snapshot) {
      return snapshot.docs.map((doc) {
        final user = doc.data();

        return user;
      }).toList();
    });
  }

  // send message
  Future<void> sendMessage(String receiverId, message) async {
    final String currentUserId = _firebaseAuth.currentUser!.uid;
    final String? current userEmail = _firebaseAuth.currentUser!.email;
    final Timestamp timestamp = Timestamp.now();
    MessageModel newMessage = MessageModel(
      senderId: currentUserId,
      senderEmail: current userEmail ?? "",
      receiverId: receiverId,
      message: message,
      timestamp: timestamp,
    );

    // construct chat room ID for the two users (sorted to ensure uniqueness)
    List<String> ids = [currentUserId, receiverId];
    ids.sort(); // sort the ids (this ensure the chatroomId is the same for any 2
people) String chatRoomID = ids.join('_');
    String chatRoomID = ids.join("_");
    // add new message to database
    await _firestore
      .collection("chat_rooms")
      .doc(chatRoomID)
      .collection("messages")
      .add(newMessage.toMap());
  }

  // get message
  Stream<QuerySnapshot> getMessage(String userID, otherUserID) {
    // construct chat room ID for the two users (sorted to ensure uniqueness)
    List<String> ids = [userID, otherUserID];
    ids.sort(); // sort the ids (this ensure the chatroomId is the same for any 2
people) String chatRoomID = ids.join('_');
    String chatRoomID = ids.join("_");

    return _firestore
      .collection("chat_rooms")
      .doc(chatRoomID)
      .collection("messages")
      .orderBy('timestamp', descending: true)
      .snapshots();
  }

  Future<String?> getUidByEmail(String email) async {
    final q = await _firestore
      .collection('Users')
      .where('email', isEqualTo: email)
      .limit(1)
      .get();
    if (q.docs.isEmpty) return null;
    // Ensure your Users doc actually stores the Firebase Auth UID in a field `uid`
    return (q.docs.first.data()['uid'] ?? '').toString();
  }
}
```



models/message.dart

```
import 'package:cloud_firestore/cloud_firestore.dart';

class MessageModel {
  final String senderId;
  final String senderEmail;
  final String receiverId;
  final String message;
  final Timestamp timestamp;

  MessageModel({
    required this.senderId,
    required this.senderEmail,
    required this.receiverId,
    required this.message,
    required this.timestamp,
  });

  factory MessageModel.fromMap(Map<String, dynamic> map) {
    return MessageModel(
      senderId: map['senderId'],
      senderEmail: map['senderEmail'],
      receiverId: map['receiverId'],
      message: map['message'],
      timestamp: map['timestamp'],
    );
  }

  Map<String, dynamic> toMap() {
    return {
      'senderId': senderId,
      'senderEmail': senderEmail,
      'receiverId': receiverId,
      'message': message,
      'timestamp': timestamp,
    };
  }
}
```



screens/chat_screen.dart

```
import 'package:flutter/material.dart';
import 'package:flutter_screenutil/flutter_screenutil.dart';

import '../services/chat_service.dart';
import '../services/user_service.dart';
import '../widgets/custom_text.dart';
import 'chat_detailscreen.dart';

class ChatScreen extends StatefulWidget {
  const ChatScreen({super.key});

  @override
  State<ChatScreen> createState() => _ChatScreenState();
}

class _ChatScreenState extends State<ChatScreen> {
  final TextEditingController _searchChatController = TextEditingController();
  final ChatService _chatService = ChatService();
  String? _currentUserEmail;
  String _searchText = '';

  @override
  void initState() {
    super.initState();
    _loadCurrentUserEmail();
  }

  Future<void> _loadCurrentUserEmail() async {
    final userData = await userService.value.getUserData();
    setState(() {
      _currentUserEmail = userData['email'];
    });
  }

  @override
  void dispose() {
    _searchChatController.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return SingleChildScrollView(
      child: Column(
        children: [
          SizedBox(height: 20.h),
          Padding(
            padding: EdgeInsets.symmetric(horizontal: 23.w),
            child: TextField(
              controller: _searchChatController,
              textInputAction: TextInputAction.search,
              decoration: InputDecoration(
                hintText: 'Search chat...',
                prefixIcon: const Icon(Icons.search),
                suffixIcon: (_searchChatController.text.isNotEmpty)
                  ? IconButton(
                      tooltip: 'Clear',
                      icon: const Icon(Icons.cancel),
                      onPressed: () {
                        setState(() {
                          _searchChatController.clear();
                          _searchText = '';
                        });
                      },
                    )
                  : null,
                border: OutlineInputBorder(
                  borderRadius: BorderRadius.circular(10),
                ),
              ),
            ),
          ),
          SizedBox(height: 10.h),
          // Users Stream
          StreamBuilder<List<Map<String, dynamic>>>(
            stream: _chatService.getUsersStream(),
            builder: (context, snapshot) {
              if (snapshot.connectionState == ConnectionState.waiting) {
                return Container(
                  height: ScreenUtil().screenHeight * 0.6,
                  padding: EdgeInsets.all(16.sp),
                  child: const Center(
                    child: CircularProgressIndicator.adaptive(),
                  ),
                );
              }
            },
          ),
        ],
      ),
    );
  }
}
```



```

        if (snapshot.hasError) {
          return Container(
            height: ScreenUtil().screenHeight * 0.6,
            padding: EdgeInsets.all(16.sp),
            child: Center(
              child: CustomText(
                text: 'Error loading users',
                fontSize: 16.sp,
              ),
            ),
          );
        }
        if (!snapshot.hasData || snapshot.data!.isEmpty) {
          return Container(
            height: ScreenUtil().screenHeight * 0.6,
            padding: EdgeInsets.all(16.sp),
            child: Center(
              child: CustomText(
                text: 'No users found',
                fontSize: 16.sp,
              ),
            ),
          );
        }

        if (users.isEmpty) {
          return Container(
            height: ScreenUtil().screenHeight * 0.6,
            padding: EdgeInsets.all(16.sp),
            child: Center(
              child: CustomText(
                text: 'No messages found...',
                fontSize: 16.sp,
              ),
            ),
          );
        }

        return ListView.builder(
          shrinkWrap: true,
          padding: EdgeInsets.symmetric(horizontal: 16.w),
          physics: const NeverScrollableScrollPhysics(),
          itemCount: users.length,
          itemBuilder: (context, index) {
            final user = users[index];
            return GestureDetector(
              onTap: () {
                Navigator.push(
                  context,
                  MaterialPageRoute(
                    builder: (context) => ChatDetailScreen(
                      currentUserEmail: _currentUserEmail!,
                      tappedUser: user,
                    ),
                  ),
                );
              },
              child: Card(
                child: ListTile(
                  leading: CircleAvatar(
                    child: CustomText(
                      text:
                        user['firstName'] != null &&
                        user['firstName'].toString().isNotEmpty
                        ? user['firstName'][0].toUpperCase()
                        : '?',
                      fontSize: 16,
                      fontWeight: FontWeight.bold,
                    ),
                  ),
                  title: CustomText(
                    text: user['firstName'] ?? 'Unknown',
                    fontSize: 20,
                    fontWeight: FontWeight.bold,
                  ),
                  subtitle: CustomText(
                    text: user['email'] ?? 'No email',
                    fontSize: 12,
                    fontWeight: FontWeight.w300,
                  ),
                ),
              ),
            );
          },
        );
      },
    ),
  ),
);
}
}

```



chat_detailscreen.dart

```
import 'package:cloud_firestore/cloud_firestore.dart';
import 'package:flutter/material.dart';
import 'package:flutter_screenutil/flutter_screenutil.dart';
import 'package:robles_advmobprog/widgets/custom_text.dart';

import '../services/chat_service.dart';
import '../services/user_service.dart';

final ChatService chatService = ChatService();

class ChatDetailScreen extends StatefulWidget {
  final String currentUserEmail;
  final Map<String, dynamic> tappedUser;

  const ChatDetailScreen({
    Key? key,
    required this.currentUserEmail,
    required this.tappedUser,
  }) : super(key: key);

  @override
  State<ChatDetailScreen> createState() => _ChatDetailScreenState();
}

class _ChatDetailScreenState extends State<ChatDetailScreen> {
  final TextEditingController _msgCtrl = TextEditingController();
  final FocusNode _msgFocus = FocusNode();
  final ScrollController _scrollCtrl = ScrollController();

  late Future<String> _currentUserIdFuture;
  bool _isSending = false;
  Timestamp? _sendingStartedAt;

  static const _postSendDelay = Duration(milliseconds: 600);

  @override
  void initState() {
    super.initState();
    _currentUserIdFuture = _getCurrentUserId();
  }

  Future<String> _getCurrentUserId() async {
    final userData = await userService.value.getUserData();
    return (userData['uid'] ?? '').toString();
  }

  @override
  void dispose() {
    _msgCtrl.dispose();
    _msgFocus.dispose();
    _scrollCtrl.dispose();
    super.dispose();
  }

  Future<void> _send(String currentUserId, String receiverId) async {
    final text = _msgCtrl.text.trim();
    if (text.isEmpty || _isSending) return;

    setState(() {
      _isSending = true;
      _sendingStartedAt = Timestamp.now();
    });

    try {
      await chatService.sendMessage(receiverId, text);
      _msgCtrl.clear();
      _msgFocus.requestFocus();

      if (_scrollCtrl.hasClients) {
        _scrollCtrl.animateTo(
          0.0,
          duration: const Duration(milliseconds: 200),
          curve: Curves.easeOut,
        );
      }

      await Future.delayed(_postSendDelay);
    } catch (e) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text('Failed to send: $e')),
      );
    } finally {
      if (mounted) {
        setState(() {
          _isSending = false;
          _sendingStartedAt = null;
        });
      }
    }
  }

  @override
  Widget build(BuildContext context) {
    final tappedUserId = (widget.tappedUser['uid'] ?? '').toString();
    final tappedUserName = (widget.tappedUser['firstName'] ?? '').toString();

    return FutureBuilder<String>(
      future: _currentUserIdFuture,
      builder: (context, snap) {
        if (snap.connectionState == ConnectionState.waiting) {
          return const Scaffold(
            body: Center(child: CircularProgressIndicator()),
          );
        }
        if (snap.hasError || !snap.hasData || snap.data!.isEmpty) {
          return const Scaffold(
            body: Center(child: Text('Error loading user data')),
          );
        }

        final currentUserId = snap.data!;

        return Scaffold(
          appBar: AppBar(
            centerTitle: true,
            title: CustomText(text: tappedUserName, fontSize: 25.sp),
          ),
          body: Column(
            children: [
              // Messages
              Expanded(
                child: StreamBuilder<QuerySnapshot>(
                  stream: chatService.getMessage(currentUserId, tappedUserId),
                  builder: (context, snapshot) {
                    if (snapshot.connectionState == ConnectionState.waiting) {
                      return const Center(child: CircularProgressIndicator());
                    }
                    if (snapshot.hasError) {
                      return Center(
                        child: Text('Error loading messages: ${snapshot.error}'),
                      );
                    }
                  },
                ),
              ),
            ],
          ),
        );
      },
    );
  }
}
```



```
List<QueryDocumentSnapshot> docs =
  snapshot.data?.docs ?? [];

// hide my just-sent messages until delay completes
if (!_isSending && _sendingStartedAt != null) {
  docs = docs.where((d) {
    final data = d.data() as Map<String, dynamic>;
    final senderId = (data['senderId'] ?? '').toString();
    final ts = data['timestamp'];
    if (senderId != currentUserId) return true;
    if (ts is Timestamp) {
      return ts.compareTo(_sendingStartedAt!) < 0;
    }
    return true;
  }).toList();
}

if (docs.isEmpty) {
  return const Center(child: Text('No messages yet'));
}

return ListView.builder(
  controller: _scrollCtrl,
  reverse: true,
  padding: const EdgeInsets.symmetric(vertical: 8),
  itemCount: docs.length,
  itemBuilder: (context, index) {
    final data =
      docs[index].data() as Map<String, dynamic>;
    final msgText = (data['message'] ?? '').toString();
    final senderId = (data['senderId'] ?? '').toString();
    final isMe = senderId == currentUserId;

    return Align(
      alignment:
        isMe ? Alignment.centerRight : Alignment.centerLeft,
      child: Container(
        margin: const EdgeInsets.symmetric(
          vertical: 4, horizontal: 8),
        padding: const EdgeInsets.symmetric(
          vertical: 10, horizontal: 12),
        constraints: BoxConstraints(
          maxWidth:
            MediaQuery.of(context).size.width * 0.75,
        ),
        decoration: BoxDecoration(
          color: isMe
            ? Theme.of(context)
              .colorScheme
                .primary
                  .withOpacity(0.5)
                : Colors.grey.shade300,
          borderRadius: BorderRadius.circular(12).copyWith(
            bottomLeft: isMe
              ? const Radius.circular(12)
                : Radius.zero,
            bottomRight: isMe
              ? Radius.zero
                : const Radius.circular(12),
          ),
        ),
        child: CustomText(
          text: msgText.isEmpty ? msgText : "[empty]",
          fontSize: 15.sp,
          fontWeight: FontWeight.normal,
          textAlign: TextAlign.left,
        ),
      ),
    );
  },
);

// Composer
SafeArea(
  top: false,
  child: Padding(
    padding: const EdgeInsets.fromLTRB(8, 0, 8, 8),
    child: Row(
      children: [
        Expanded(
          child: TextField(
            controller: _msgCtrl,
            focusNode: _msgFocus,
            enabled: !_isSending,
            textInputAction: TextInputAction.send,
            minLines: 1,
            maxLines: 4,
            onSubmitted: (_) =>
              !_isSending ? _send(currentUserId, tappedUserId) : null,
            decoration: const InputDecoration(
              hintText: 'Type a message...',
              hintStyle: TextStyle(fontFamily: 'Poppins'),
              border: OutlineInputBorder(),
              isDense: true,
            ),
          ),
        ),
        const SizedBox(width: 8),
        !_isSending
          ? const SizedBox(
              height: 24,
              width: 24,
              child: CircularProgressIndicator(strokeWidth: 2),
            )
          : IconButton(
              icon: const Icon(Icons.send),
              onPressed: () => _send(currentUserId, tappedUserId),
            ),
      ],
    ),
  ),
);
```

Additional Instructions:

Using the FloatingAction button redirect the Chat List UI.

Enhancement 1: Chat List – User Display

- Display all registered users in the chat list.
- Exclude the **current logged-in user** from the list.

Enhancement 2: Chat List – Search Functionality

- Add a **search bar** at the top of the chat list.
- Enable users to quickly **filter and locate other users** by typing their name or email.

Enhancement 3: Chat Detail Screen – Redesign

- Apply a **modern redesign** with smooth animations.
- Improve **chat bubble design** for clearer sender/receiver distinction.
- Show **sending states** (e.g., “sending...”, checkmarks for delivered/seen).
- Incorporate **UI animations** for message transitions (fade/slide).

Laboratory Discussion: *Discuss how’s the user structure in each collection of the **cloud_firestore** when initiating a chat. In addition what will happen if you’ve initiate your own chat or you chat yourself.*

Git Instruction:

Update first the README.md file adding the discussion part for example:

Lab Activity 6: *discussion*

after the hands-on and updating the README.md (must be on root folder surname-advmobprogAY2627)

1. `git checkout -b lab_act6`
2. `git add .`
3. `git commit -m “lab_act6”`
4. `git push origin lab_act6`

Activity Rubrics

Criteria	Excellent (4 pts)	Good (3 pts)	Fair (2 pts)	Poor (1 pt)	Points
Code Implementation	Code runs flawlessly, includes all required features (placeholders, avatars, widgets for like/comment).	Code mostly correct, minor errors, or missing one minor feature.	Noticeable errors or missing two or more features.	Code does not run or is significantly incomplete.	
Design Consistency	UI closely resembles intended layout, proper spacing, alignment, and styling.	Layout mostly accurate, minor spacing or alignment issues.	Layout deviates significantly from the intended design or appears incomplete.	Layout is unrecognizable or does not follow the instructions.	
Use of Widgets and Enhancements	Properly implements widgets for placeholders, avatars, and like/comment buttons.	Most widgets are correctly implemented, with minor issues or missing one enhancement.	Widgets are inconsistently used or missing two or more enhancements.	Widgets are not used, or enhancements are not implemented.	
Code Organization and Readability	Code is well-structured, follows best practices, and adheres to the specified file organization.	Code is mostly organized and readable, minor deviations from the required structure.	Code is poorly organized or does not follow the specified structure.	Code is disorganized and difficult to understand.	
Back-End Integration	Back-end integration works flawlessly with efficient API calls, secure data handling, and real-time updates; no errors or downtime.	Back-end integration is mostly correct with only minor API, data, or response handling issues.	Back-end integration has noticeable flaws or inefficiencies; multiple API calls fail or data handling is inconsistent.	Back-end integration has major issues; frequent failures, poor response handling, or insecure practices.	
Other Comments/ Observations:				Total Score	
				Rating: (Total Score/20) * 100	